

Fiche 8 : mise en place d'un micro-contrôleur ESP 8266

Contexte :

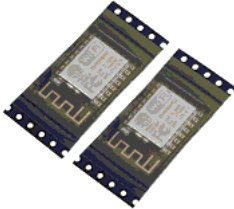
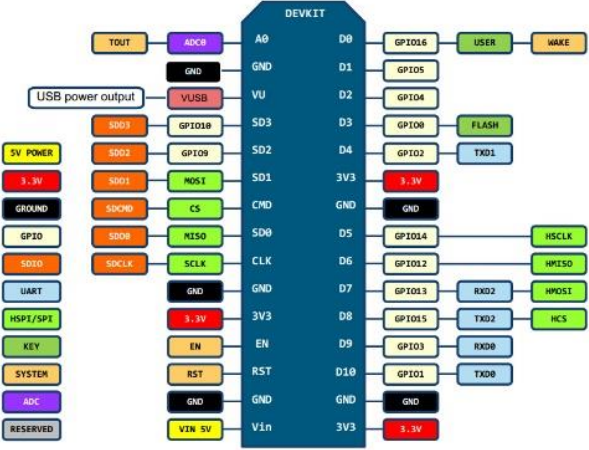
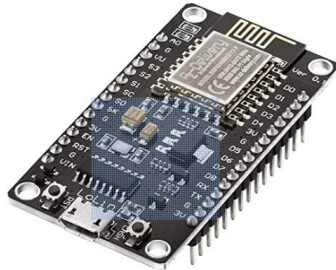
Paramétrer Arduino pour programmer un micro-contrôleur ESP 8266

1 DESCRIPTION DE LA CARTE NODEMCU BASEE SUR L'ESP8266

Dans la première partie du cours, nous avons construit une connexion client-serveur sous python. L'objectif est ici de montrer que la connexion entre un client et un serveur peut se faire indépendamment du langage employé et du système matériel utilisé.

Le serveur sera placé dans un microcontrôleur (ESP8266) et le client restera le PC avec un programme en python. Cette puce a été développée en 2014 par la société **Chinoise Espressif** d'où le nom ESP.

L'ESP devra être préalablement connecté au réseau wifi.

Le module radiofréquence : ESP8266 (µc+RF)	Schéma électrique des connexions	Carte nodeMCU (8 euros) (ESP8266+USB+firmware) connecte le µc à un PC en USB, fournit un firmware de programmation du µc
 figure 1 : module RF avec son antenne MLA Mender Line Antenna (meander signifie sinueux, méandre).	ESP8266 – LoLin www.wemos.cc/wiki/  DEVKIT Une LED bleue est connectée en interne sur D4 figure 2 : Pin de connexion du module LOLIN	 Figure 3 : photo de la carte LOLIN munie de son port micro USB

<http://framboiseaupotager.blogspot.com/2017/12/lessentiel-sur-esp8266-nodemcu-sequence.html>

Site qui donne des infos sur les bogues ESP :

https://sigmdel.ca/michel/program/esp8266/arduino/watchdogs_fr.html

https://sigmdel.ca/michel/program/esp8266/arduino/watchdogs2_fr.html

Description:

Modèle: ESP8266-12 ISM 2.4 GHz PA + 25dBm 802.11b/g/n

L'ESP8266 peut se programmer de plusieurs façons :

- Avec des scripts Lua, interprétés ou compilés, avec le firmware [NodeMCU](#)
- En C++, avec l'IDE [Arduino](#) (notre choix)
- En JavaScript, avec le firmware [Espruino](#)
- En MicroPython, avec le firmware [MicroPython](#)
- En C, avec le SDK (Software development kit) d'Espressif

La prise en main se fera à travers un programme simple :

- Programme « hello word »

Lorsque le programme est compilé, il faut l'injecter dans le microcontrôleur. On appelle cela flasher le composant car notre programme deviendra le firmware de ce dernier. Normalement, la carte NodeMCU se met automatiquement en mode apprentissage et normalement il n'y a rien à faire avant de téléverser le programme depuis l'IDE Arduino. Dans la pratique ceci ne fonctionne pas toujours. Cela dépend de ce qui est connecté aux PINs et aussi si c'est la première fois que le composant est flashé.

Quand le « flashage » de la carte échoue, il faut remettre la carte dans le mode apprentissage en utilisant les boutons situés de part et d'autre du port micro USB comme indiqué ci-dessous :

- Appuyez sur le bouton Flash et maintenez le bouton appuyé.
- Appuyez sur Reset
- Relâchez le bouton Reset
- Relâchez le bouton Flash

2 PARAMETRAGE ARDUINO

Cas particulier de la salle 203-202-201 de RT avec le portail captif

Il faudra dans « préférences » et l'onglet « Réseau » puis cocher configuration manuelle du proxy. Dans Nom d'hôte mettre « webcache » et dans numéro de port mettre « 3128 »

Connecter l'ESP8266 sur le port USB, ouvrir l'IDE Arduino.

Dans le menu « Outils » => « port », choisir le port de connexion.

Dans le menu « fichier » => « préférences », vérifier que l'URL de gestion des cartes est bien http://arduino.esp8266.com/stable/package_esp8266com_index.json

Dans le menu « outils » => « type de carte » => choisir « NodeMCU 1.0 (ESP-12E module) »

Ouvrir la console de visualisation des messages dans le menu « outils » => « moniteur série » . Choisir la configuration suivante « NL et CR » et une vitesse « 115200 baud »

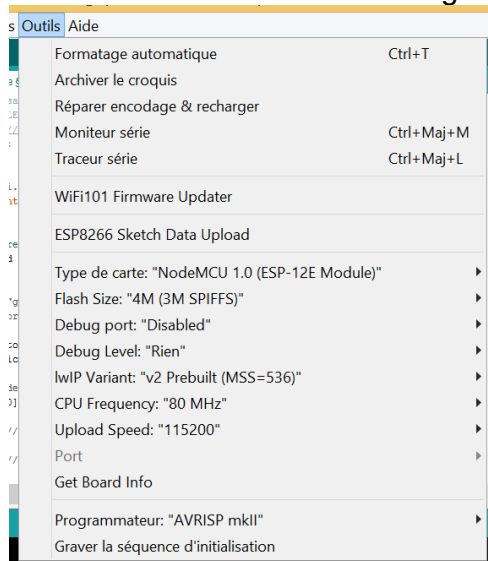
Dans le menu « outils » => « Flash Size » => choisir « 4M(3M SPIFFS) »

Dans le menu « outils » => « Upload Speed » => choisir « 115200 »

Dans le menu « outils » => « CPU frequency » => choisir « 80MHz »

Dans le menu « outils » => « port » => choisir le bon « tty »

Vous devrez être dans la configuration suivante :



2.1 PROGRAMME 1 "HELLO WORD"

Nous allons créer un programme.

Dans le menu « fichier » => « Nouveau ». Avant la fonction setup(), placer la ligne suivante :

```
int i=0;
```

Dans la fonction setup(), placer les 2 lignes suivantes :

```
Serial.begin(115200);  
Serial.println("Hello Word");
```

Dans la fonction loop(), placer les 2 lignes suivantes :

```
Serial.printf("valeur=%d\n",i++);  
delay(1000);
```

Compiler et téléverser le programme. Vérifier avec la console que le programme affiche « Hello Word » puis une valeur qui croit chaque seconde.

2.2 PROGRAMME 2: "SERVEUR WEB"

Dans le menu « fichier » => « exemples » => « ESP8266WebServer » puis choisir le fichier « HelloServer »

Compiler et téléverser le programme. Vérifier avec la console que le programme donne l'adresse IP du serveur web.

EDITION DU PROGRAMME « HelloServer »

(attention, qq modifications par rapport au fichier d'origine)

```
#include <ESP8266WiFi.h>  
#include <WiFiClient.h>  
#include <ESP8266WebServer.h>  
#include <ESP8266mDNS.h>  
  
const char* ssid = "votre_RasPwnOS_X";  
const char* password = "ln53cur3!";  
  
ESP8266WebServer server(80);
```

```

const int led = 13;

void handleRoot() {
  digitalWrite(led, 1);
  server.send(200, "text/plain", "hello from esp8266 and binome X !");
  digitalWrite(led, 0);
}

void handleNotFound() {
  digitalWrite(led, 1);
  String message = "File Not Found\n\n";
  message += "URI: ";
  message += server.uri();
  message += "\nMethod: ";
  message += (server.method() == HTTP_GET) ? "GET" : "POST";
  message += "\nArguments: ";
  message += server.args();
  message += "\n";
  for (uint8_t i = 0; i < server.args(); i++) {
    message += " " + server.argName(i) + ": " + server.arg(i) + "\n";
  }
  server.send(404, "text/plain", message);
  digitalWrite(led, 0);
}

void setup(void) {
  pinMode(led, OUTPUT);
  digitalWrite(led, 0);
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.println("");

  // Wait for connection
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to ");
  Serial.println(ssid);
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

```

```

if (MDNS.begin("esp8266")) {
  Serial.println("MDNS responder started");
}

server.on("/", handleRoot);

server.on("/inline", []() {
  server.send(200, "text/plain", "this works as well");
});

server.on("/gif", []() {
  static const uint8_t gif[] PROGMEM = {
    0x47, 0x49, 0x46, 0x38, 0x37, 0x61, 0x10, 0x00, 0x10, 0x00, 0x80, 0x01,
    0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0x2c, 0x00, 0x00, 0x00, 0x00,
    0x10, 0x00, 0x10, 0x00, 0x00, 0x02, 0x19, 0x8c, 0x8f, 0xa9, 0xcb, 0x9d,
    0x00, 0x5f, 0x74, 0xb4, 0x56, 0xb0, 0xb0, 0xd2, 0xf2, 0x35, 0x1e, 0x4c,
    0x0c, 0x24, 0x5a, 0xe6, 0x89, 0xa6, 0x4d, 0x01, 0x00, 0x3b
  };
  char gif_colored[sizeof(gif)];
  memcpy_P(gif_colored, gif, sizeof(gif));
  // Set the background to a random set of colors
  gif_colored[16] = millis() % 256;
  gif_colored[17] = millis() % 256;
  gif_colored[18] = millis() % 256;
  server.send(200, "image/gif", gif_colored, sizeof(gif_colored));
});

server.onNotFound(handleNotFound);

server.begin();
Serial.println("HTTP server started");
}

void loop(void) {
  server.handleClient();
  MDNS.update();
}

```